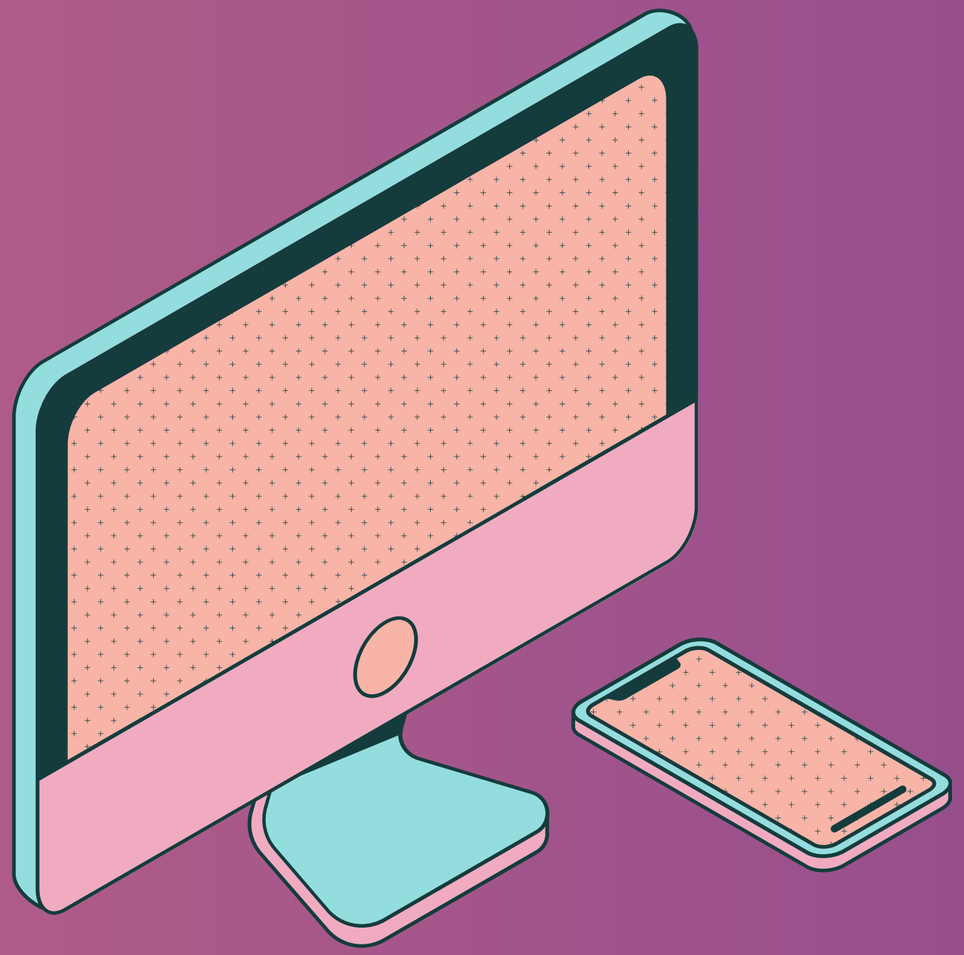
Hooks React Native

Hooks essenciais do React Native

Feito por: Ana Katy

Introdução

Com a evolução do ecossistema React, os componentes funcionais tornaram-se o padrão para o desenvolvimento moderno, substituindo progressivamente os componentes de classe. O principal motor desta mudança foi a introdução dos Hooks. Esta pesquisa explora os fundamentos dos Hooks no contexto do React Native, detalhando como ferramentas como useState, useEffect e useRef permitem gerir estados, ciclos de vida e referências de forma eficiente e declarativa, seguindo as melhores práticas recomendadas pela documentação oficial.



O que é um hook? Para que serve?

Definição: Hooks são funções especiais que permitem "conectar" (hook into) os recursos de estado e ciclo de vida do React dentro de componentes funcionais.

Para que servem: Antes dos hooks, recursos como o estado interno (state) só estavam disponíveis em componentes de classe. Os hooks permitem que você use o React sem classes, facilitando o compartilhamento de lógica entre componentes, tornando o código mais limpo, legível e fácil de testar.

Explicação dos Hooks, Uso e Exemplos

1. useState

- O que faz: Adiciona o estado local ao componente. Ele retorna um par: o valor do estado atual e uma função que permite atualizá-lo.
- Quando usar: Sempre que você precisar que o componente "lembre" de alguma informação que pode mudar e que deve disparar uma nova renderização na tela (ex: texto de um input, contador, dados de uma API).

```
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <button onClick={handleClick}>
      You pressed me {count} times
    </button>
  );
}

```

Explicação dos Hooks, Uso e Exemplos

2. useEffect

- O que faz: Permite executar efeitos colaterais (side effects) em componentes funcionais. Ele substitui métodos de ciclo de vida como componentDidMount, componentDidUpdate e componentWillUnmount.

- Quando usar: Para chamadas de API, assinaturas de eventos (listeners), manipulação direta do DOM/Native UI ou timers.

```react/ModalDialog.jsx
import { useEffect, useRef } from 'react';

export default function ModalDialog({ isOpen, children }) {
  const ref = useRef();

  useEffect(() => {
    if (!isOpen) {
      return;
    }
    const dialog = ref.current;
    dialog.showModal();
    return () => {
      dialog.close();
    };
  }, [isOpen]);

  return <dialog ref={ref}>{children}</dialog>;
}
```

Explicação dos Hooks, Uso e Exemplos

3. useRef

- O que faz: Retorna um objeto mutável cuja propriedade .current persiste durante todo o ciclo de vida do componente. Diferente do useState, alterar um ref **não** causa uma nova renderização.
- Quando usar: Para acessar diretamente uma instância de um componente (ex: focar um input) ou para armazenar qualquer valor que precise ser persistido mas que não afete visualmente o componente.

```
import { useRef } from 'react';

export default function Counter() {
  let ref = useRef(0);

  function handleClick() {
    ref.current = ref.current + 1;
    alert('You clicked ' + ref.current + ' times!');
  }

  return (
    <button onClick={handleClick}>
      Click me!
    </button>
  );
}
```

O array de dependências no useEffect

O comportamento do useEffect muda drasticamente dependendo do segundo argumento:

1. Array Vazio **[]**: O efeito é executado apenas uma vez, logo após a montagem do componente (similar ao componentDidMount). É ideal para inicializações e chamadas de API únicas.
2. Sem nenhum array: O efeito é executado após cada renderização do componente. Isso pode causar problemas de performance ou loops infinitos se você atualizar o estado dentro do efeito.
3. Com variáveis **[dep1, dep2]**: O efeito é executado na montagem e sempre que qualquer uma das variáveis dentro do array sofrer uma alteração.

useRef vs useState

Característica	useState	useRef
Renderização	Atualizar o estado causa re-renderização.	Atualizar a ref **não** causa re-renderização.
Persistência	O valor é preservado entre renders.	O valor é preservado entre renders.
Acesso	Acesso direto à variável.	Acesso via propriedade .current.

Utilidade especial no React Native: O useRef é essencial para lidar com instâncias de componentes nativos, como:

- Controlar a reprodução de um vídeo ou áudio.
- Manipular programas de animação (Animated.Value).
- Acessar métodos de um ScrollView (ex: scrollTo).
- Gerenciar o foco de campos de formulário.

Regras dos Hooks (Rules of Hooks)

1. Apenas chame Hooks no nível superior (Top Level): Não chame hooks dentro de loops, condições (if) ou funções aninhadas.
2. Apenas chame Hooks em funções React: Use apenas em componentes funcionais ou hooks customizados.

Por que não dentro de um if?

O React depende da ordem das chamadas para associar o estado correto. Se um if pular um hook, a ordem quebra e o React pode atribuir o estado de uma variável a outra, causando bugs imprevisíveis.

Conclusão

Compreender o funcionamento dos Hooks é fundamental para dominar o desenvolvimento em React Native. Enquanto o useState garante a reatividade da interface, o useEffect gere a interação com o mundo externo e o useRef oferece um escape para manipulações técnicas e imperativas. Ao respeitar as "Regras dos Hooks", garantimos que a aplicação seja estável, performática e fácil de manter, aproveitando todo o potencial da programação declarativa proposta pelo React.

Fim, obrigada por asssistir